CampusCare – Antigravity Development Prompt

A complete project-context prompt for developing the frontend and integrating it with the completed Spring Boot backend.

How to use this PDF

Open your CampusCare project in Antigravity, upload/provide the completed backend source code, then paste the prompt below. Tell Antigravity to inspect the existing backend before changing anything. The prompt is intentionally detailed so it understands the project goals, architecture, privacy requirements, UI expectations, and development workflow.

MASTER PROMPT

```
You are the lead frontend engineer and integration developer for my college project named:

CAMPUSCARE
“A Centralized Anonymous Feedback and Complaint Management System for Hostel and Mess”

IMPORTANT:
I have already completed the backend. I will provide/upload my existing backend source code in this workspace

================================================
1. PROJECT PURPOSE
================================================

CampusCare is a web-based hostel and mess complaint/feedback management system.

The main problem:
Students often report hostel/mess problems verbally, through scattered messages, or through informal channels

CampusCare provides one centralized platform where verified students can submit complaints or feedback, while

The system should focus on:
- Anonymous student feedback
- Genuine/verified student access
- Complaint tracking
- Fake/spam/suspicious feedback detection
- Similar complaint grouping
- Admin accountability
- Automatic reminders
- Escalation of overdue complaints
- Resolution proof
- Student confirmation before final closure
- Useful analytics

Do not make unrealistic claims such as “the system can perfectly detect lies.” Suspicious feedback should be

================================================
2. TECHNOLOGY
================================================

Existing backend:
- Java 25
- Spring Boot
- MySQL
- Spring Data JPA / REST APIs as implemented in the uploaded backend
- Backend developed/tested in VS Code

Frontend:
- Use a clean modern web frontend.
- Prefer HTML, CSS and JavaScript unless the existing project already requires another frontend framework.
- Do not introduce unnecessary frameworks or dependencies.
```

- Keep the code understandable for a college student.

The frontend must communicate with the backend through REST APIs.

==========
3. FIRST TASK — INSPECT BEFORE CODING
==========

Before writing frontend code:

1. Inspect every backend source file.
2. Identify the Spring Boot main class.
3. Identify all controllers and endpoints.
4. Identify HTTP methods: GET, POST, PUT, DELETE, etc.
5. Identify request bodies and response JSON structures.
6. Identify entity/model fields.
7. Identify status values and enums.
8. Identify authentication/login endpoints if present.
9. Identify student verification logic if present.
10. Identify admin authentication and roles if present.
11. Identify file/photo upload endpoints if present.
12. Identify reminder/escalation endpoints if present.
13. Identify validation/fake-feedback detection logic if present.
14. Identify CORS configuration.
15. Identify the backend base URL and port if available.

Create an internal API map before building the UI.

IMPORTANT:
Do not invent API endpoints when an equivalent endpoint already exists in the backend. Use the exact backend

If something is missing from the backend, clearly identify it and build the frontend so that it can be connec

==========
4. FRONTEND PAGES
==========

Build a professional, responsive CampusCare frontend with these main areas:

A. Landing/Home Page
- CampusCare logo/name
- Short explanation
- “Student Login”
- “Admin Login”
- “How it works”
- Key features
- Privacy/anonymous feedback explanation

B. Student Login / Verification
- College email input
- Verification/OTP UI if supported by backend
- Clear validation messages
- Do not expose private student information unnecessarily

C. Student Dashboard
Show:
- Welcome message
- Submit Complaint
- My Complaints
- Complaint status
- Notifications/reminders
- Profile/verification status if supported

D. Submit Complaint Page
Fields should match the backend:
- Category
- Location/hostel/block

- Description
- Priority only if the backend allows student selection
- Photo/evidence upload if supported

Use category-based evidence:
- Physical problems such as water leakage, broken equipment, dirty areas can request/require evidence if the
- Experience-based problems such as food quality, Wi-Fi, or staff behavior should not blindly require a photo

E. Complaint Tracking
Show:
- Complaint ID / anonymous reference
- Category
- Location
- Date
- Current status
- Priority
- Timeline
- Reminder/overdue state if available

Do not expose the student's identity to administrators through ordinary complaint views.

F. Student Resolution Confirmation
When the admin marks a complaint resolved:
- Show the resolution information
- Ask:
  “Is your problem actually solved?”
- YES → close/confirm if backend supports it
- NO → reopen/escalate if backend supports it
- Explain the result clearly to the student

G. Admin Login
- Secure login UI
- Do not store plaintext passwords in frontend code
- Respect backend authentication

H. Admin Dashboard
Create a professional dashboard containing:
- Total complaints
- Pending
- In Progress
- Resolved
- Overdue
- Suspicious / Needs Review
- High priority issues
- Recent complaints

Add filters:
- Category
- Location
- Status
- Priority
- Date
- Validation status

I. Admin Complaint Details
Show:
- Complaint information
- Anonymous student/reference where appropriate
- Evidence/photo
- Similar complaint count if available
- Validation/suspicion result if available
- Deadline
- Reminder history if available
- Escalation status
- Resolution notes
- Resolution proof
- Actions available to the admin

J. Analytics
Show useful charts/cards if the backend provides the required data:
- Complaints by category
- Complaints by hostel/location
- Status distribution
- Resolution time
- Recurring problems

Do not create fake statistics. If the backend does not provide real analytics data, display only data that ca

==========
5. UNIQUE CAMPUSCARE FEATURES
==========

A. Anonymous Feedback
The student can be verified for access while the complaint shown to normal administrators remains anonymous.

Do not expose the student's email in the complaint UI unless the backend explicitly requires an authorized wo

B. Fake/Spam/Suspicious Feedback Detection
The backend may use:
- Spam checks
- Duplicate checks
- Abusive language checks
- Submission rate limits
- Suspicion score

The frontend should display results as:
- Normal
- Needs Review
- Suspicious

Do NOT say “this student is lying.”

Use neutral language:
“Feedback flagged for review.”

C. Similar Complaint Grouping
If the backend identifies similar complaints:
- Show “Similar reports: X”
- Group related issues visually
- Help the admin understand recurring problems

D. Smart Accountability & Escalation
Show:
- Assigned administrator/authority where permitted
- Deadline
- Remaining time
- Reminder status
- Overdue status
- Escalation level

Concept:
Complaint → assigned → reminder → overdue → escalation → resolution → student confirmation.

The frontend should make this process easy to understand.

E. Resolution Proof
If supported:
- Admin enters action taken
- Admin adds resolution note
- Admin uploads proof/photo
- Student later confirms whether the problem was actually solved

==========
6. ANONYMITY AND PRIVACY
==========

This is very important.

The system should distinguish:
1. Identity verification
2. Complaint anonymity

A student may need to verify that they are a genuine college student, but their identity should not be casua

Never display:
- Student email
- Password
- OTP
- Sensitive authentication data

unless required by an authorized screen and the backend contract.

Use an anonymous reference/complaint ID where appropriate.

Do not put private identity data into localStorage unless absolutely necessary.

==
7. UI / UX DESIGN
==

Design should look like a real modern college management product, not a basic student HTML assignment.

Style:
- Clean
- Professional
- Modern
- Simple
- Responsive
- Easy for students and administrators
- Good spacing
- Clear typography
- Accessible buttons
- Proper loading states
- Empty states
- Error states
- Success notifications
- Confirmation dialogs for destructive actions

Suggested visual language:
- Blue/indigo primary theme
- White/light cards
- Subtle shadows
- Rounded corners
- Status badges
- Clean dashboard cards

Do not overuse animations.

Make the interface work well on:
- Desktop
- Laptop
- Tablet
- Mobile

==
8. ERROR HANDLING
==

Never show raw backend errors directly to users.

Convert them into understandable messages.

Examples:

- Login failed → “Email or password is incorrect.”
- Backend unavailable → “CampusCare server is currently unavailable. Please try again.”
- Validation error → Show the exact field that needs correction.
- Complaint submission failed → “We could not submit your complaint. Please try again.”
- Unauthorized → Redirect to login where appropriate.

Add loading indicators for API calls.

Prevent double submission by disabling the submit button while a request is in progress.

==========
9. API INTEGRATION RULES
==========

Create one clear API configuration area, for example:

API_BASE_URL

Do not scatter backend URLs throughout many files.

Create reusable JavaScript functions such as:
- apiGet()
- apiPost()
- apiPut()
- apiDelete()
- apiUpload() if needed

Handle:
- JSON requests
- JSON responses
- HTTP errors
- authentication headers if required
- multipart/form-data for file upload if required

Use the backend's actual field names.

Do not assume:
- “status = RESOLVED”
- “priority = HIGH”
- endpoint names
unless they are confirmed from the uploaded backend.

==========
10. SECURITY
==========

Follow basic secure development practices.

- Never hard-code passwords.
- Never hard-code admin credentials.
- Never expose database credentials in frontend files.
- Never store plaintext passwords.
- Do not trust frontend validation alone.
- Backend remains responsible for authorization and validation.
- Protect admin-only pages.
- Do not allow a normal student to call admin functions through the UI.
- Avoid unsafe HTML rendering of user-generated complaint text.
- Escape/sanitize user content before displaying it.

==========
11. DEVELOPMENT WORKFLOW
==========

Work incrementally.

Do NOT generate the entire frontend in one giant response.

Use this order:

STEP 1
Inspect backend and create API map.

STEP 2
Create frontend folder/file structure.

STEP 3
Build landing page.

STEP 4
Build student authentication/verification UI.

STEP 5
Connect student dashboard to real backend APIs.

STEP 6
Build complaint submission and test POST API.

STEP 7
Build complaint tracking.

STEP 8
Build admin login and dashboard.

STEP 9
Connect status update APIs.

STEP 10
Implement validation/suspicious feedback UI based on actual backend.

STEP 11
Implement reminders and escalation UI based on actual backend.

STEP 12
Implement resolution proof and student confirmation.

STEP 13
Implement analytics using real backend data.

STEP 14
Perform full integration testing.

STEP 15
Fix UI/UX issues and prepare final demo.

After every major step:
- Explain what was changed.
- List files created/modified.
- Explain how to run it.
- Explain how to test it.
- Report any backend changes required separately.

==========
12. IMPORTANT RULES FOR ANTIGRAVITY
==========

1. Do not rewrite working backend code unnecessarily.
2. Do not delete existing backend functionality.
3. Do not invent API endpoints.
4. Do not invent database fields.
5. Inspect existing code before making assumptions.
6. Reuse existing backend APIs whenever possible.
7. Keep frontend and backend loosely coupled.
8. Keep code organized and readable.
9. Do not add unnecessary dependencies.
10. Do not create fake data for final functionality.
11. Use mock data only temporarily for UI development and clearly mark it.

12. Do not claim a feature is complete until it works with the real backend.
13. If an API is missing, tell me exactly what is missing before changing backend code.
14. Preserve the anonymity/privacy concept.
15. Do not automatically label a complaint as “fake”; use “Suspicious” or “Needs Review.”
16. Do not expose private student information in admin complaint screens.
17. Do not build dangerous or unethical features intended to harass or personally punish administrators.
18. Accountability should be implemented through deadlines, reminders, audit trails, and authorized escalatio

==========
13. FINAL EXPECTED RESULT
==========

The finished CampusCare website should allow:

STUDENT:
Login/verification
→ Submit anonymous complaint
→ Add category/location/details
→ Add evidence where appropriate
→ Track complaint
→ Receive updates
→ Confirm or reject resolution

ADMIN:
Login
→ View complaints
→ Filter/group complaints
→ See suspicious/needs-review feedback
→ Manage status
→ See deadlines
→ Receive/trigger reminders
→ Handle escalated complaints
→ Add resolution notes/proof
→ Close complaint after proper workflow

SYSTEM:
Verification
→ Anonymous complaint handling
→ Spam/duplicate/suspicious detection
→ Similar complaint grouping
→ Priority
→ Deadline monitoring
→ Automatic reminders
→ Escalation
→ Resolution verification
→ Analytics

==========
14. FIRST ACTION
==========

DO NOT START CODING YET.

First inspect the uploaded CampusCare backend.

Then give me:

1. Backend technology summary
2. Project/package structure
3. All available API endpoints
4. Request/response format for each important endpoint
5. Authentication mechanism
6. Database entities and relationships
7. Existing complaint status values
8. Existing validation/fake-detection functionality
9. Existing reminder/escalation functionality
10. Existing file upload/evidence functionality

```
11. CORS configuration
12. Backend base URL/port
13. What frontend functionality can be connected immediately
14. What backend functionality is missing for the planned frontend

Then propose the frontend file structure.

WAIT FOR MY CONFIRMATION BEFORE MAKING MAJOR CODE CHANGES.

Remember:
The backend already exists. Your primary job is to build a polished frontend and integrate it correctly with
```

Recommended upload order

1. Upload the complete backend project folder. 2. Make sure the backend source files are included. 3. If you have API documentation, screenshots, or a database schema, provide those too. 4. Paste the MASTER PROMPT after the files are available to Antigravity. 5. Let it inspect first; do not ask it to immediately generate the entire website.

Important

The prompt deliberately tells Antigravity to inspect your actual backend instead of guessing endpoints or database fields. That is important because the frontend must match the backend you have already completed.